

Java Script Lab

Experiments & Case Studies

Arpeet Patel | PRN: 24070521218 | Symbiosis Institute of Technology, Nagpur

Experiment 1

Experiment Title

CSS Styling Methods: Internal, Inline, and External CSS

Software/Tools Required

- Web Browser (Chrome, Firefox, Edge)
- HTML Text Editor (VS Code, Sublime Text, Notepad++)

Experiment Program Code

```
<!DOCTYPE html>
<html>
<head>
  <title>CSS Types</title>

  <!-- External CSS -->
  <link rel="stylesheet" href="style.css">

  <!-- Internal CSS -->
  <style>
    body {
      font-family: Arial, sans-serif;
      background-color: #f4f4f9;
    }
    .internal-h1 { color: blue; font-size: 35px; }
    .internal-p { color: red; font-size: 18px; }
    .internal-btn {
      background-color: #3498db;
      color: white;
      padding: 10px 20px;
      border: none;
      border-radius: 5px;
    }
  </style>
</head>
<body>

  <div style="position:absolute; top:10px; right:20px;">
    <strong>Name:</strong>Arpeet Patel <br>
    <strong>PRN:</strong> 24070521218
  </div>

  <h2>1. Internal CSS</h2>
  <h1 class="internal-h1">Welcome to Internal CSS</h1>
  <p class="internal-p">This webpage is styled using Internal CSS.</p>
  <button class="internal-btn">Click Here</button>

  <hr>

  <h2>2. Inline CSS</h2>
  <h1 style="color:rgb(46, 250, 5); font-size:35px;">Welcome to Inline CSS</h1>
  <p style="color:rgb(97, 16, 226); font-size:18px;">This webpage is styled using Inline CSS.</p>
  <button style="background:#3498db; color:white; padding:10px 20px; border:none; border-radius:5px;">
    Click Here
  </button>

  <hr>
```

```

<h2>3. External CSS</h2>
<h1 class="external-h1">Welcome to External CSS</h1>
<p class="external-p">This webpage is styled using External CSS.</p>
<button class="external-btn">Click Here</button>

</body>
</html>

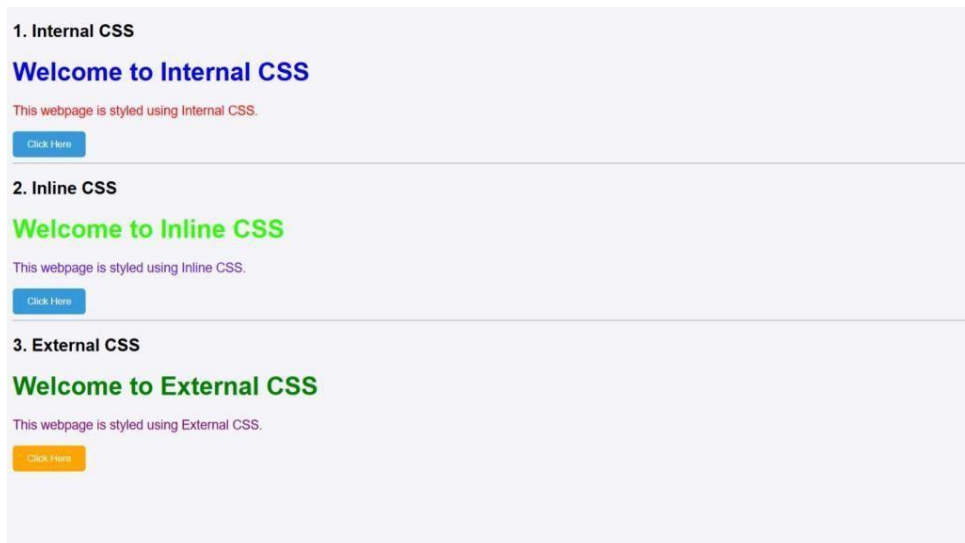
/* Style.css - External Stylesheet */
.external-h1 { color: green; font-size: 35px; }
.external-p { color: purple; font-size: 18px; }
.external-btn {
    background-color: orange;
    color: white;
    padding: 10px 20px;
    border: none;
    border-radius: 5px;
}

```

Output

The webpage displays three sections demonstrating CSS styling methods — Internal CSS (blue heading, red paragraph), Inline CSS (green heading, purple paragraph), and External CSS (green heading, purple paragraph, orange button) — each rendered with distinct styling to illustrate how the three approaches differ.

Attach the program output/screenshot.



Student Name: Arpeet Patel

PRN: 24070521218

File Path: Experiments/Exp1 (24070521218 Arpeet Patel)/Exp1.html

Result/Conclusion

The three methods of applying CSS — internal, inline, and external — were successfully implemented and verified. External CSS proved most maintainable for reusable styles, inline CSS is useful for one-off overrides, and internal CSS is convenient for page-specific styling without a separate file.

Case Study 1

Case Study Title

Interactive Academic Portal: Linking College and Student Information Systems

Aim

To design and develop a modular, client-side web application using HTML5 and JavaScript that seamlessly links global institutional profiles with dynamic, individual student data validation forms.

Software/Tools Required

- Frontend Structure: HTML5
- Client-Side Scripting: JavaScript (ES6+)
- Styling Language: CSS3
- Development Environment: Visual Studio Code / Notepad++
- Version Control: GitHub Repository System
- Execution Environment: Any modern web browser (Chrome, Firefox, Edge)

Project Description

This case study implements an interconnected web portal consisting of two components: College.html, the institutional landing page displaying general information, infrastructure, and courses offered with navigation to student directories; and StudentInfo.html, a dynamic information portal that captures, validates, and displays student data using client-side JavaScript logic.

The scripting layer enforces null-check filters on mandatory fields (PRN and Full Name), regex-based format matching for institutional identifiers, and onsubmit/onclick event handlers to process user entries interactively.

Case Study Program Code

College.html

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<title>Academic Portal - Institutional Profile</title>
<style>
  body { font-family: Arial, sans-serif; line-height: 1.6; margin: 20px; padding: 20px; background-
color: #f4f4f9; }
  header { background: #003366; color: #fff; padding: 20px; text-align: center; border-radius: 5px; }
  section { background: #fff; margin: 20px 0; padding: 20px; border-radius: 5px; box-shadow: 0 2px 5px
rgba(0,0,0,0.1); }
  h2 { color: #003366; }
  .btn { display: inline-block; background: #003366; color: #fff; padding: 10px 20px; text-decoration:
none; border-radius: 5px; font-weight: bold; }
  .btn:hover { background: #002244; }
</style>
</head>
<body>
<header>
  <h1>Welcome to the Institutional Portal</h1>
  <p>Arpeet Patel | Case Study 1</p>
</header>
<section id="about">
  <h2>About the Institution</h2>
  <p>This portal acts as the centralized academic hub managing campus infrastructure,
department assignments, and student demographic portfolios.</p>
</section>
<section id="departments">
  <h2>Courses & Departments</h2>
  <ul>
    <li>Department of Computer Science & Engineering</li>
    <li>Department of Information Technology</li>
    <li>Department of Data Science</li>
  </ul>
</section>
<section id="navigation">
  <h2>Student Information Directory</h2>
  <p>Access individual student verification forms, check submission integrity, and review registration
logs.</p>
</section>
</body>
</html>
```

```

    <a href="StudentInfo.html" class="btn">Open Student Information Portal &rarr;</a>
</section>
</body>
</html>

```

StudentInfo.html

```

<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<title>Student Information Validation Portal</title>
<style>
    body { font-family: Arial, sans-serif; line-height: 1.6; margin: 20px; padding: 20px; background-color: #f4f4f9; }
    .container { max-width: 600px; background: #fff; margin: auto; padding: 20px; border-radius: 5px; box-shadow: 0 2px 5px rgba(0,0,0,0.1); }
}
    h2 { color: #003366; text-align: center; }
    .form-group { margin-bottom: 15px; }
    label { display: block; margin-bottom: 5px; font-weight: bold; }
    input[type="text"], select { width: 100%; padding: 8px; box-sizing: border-box; border: 1px solid #ccc; border-radius: 4px; }
    button { background: #003366; color: white; padding: 10px 15px; border: none; border-radius: 4px; cursor: pointer; width: 100%; font-size: 16px; }
    button:hover { background: #002244; }
    #outputWindow { margin-top: 20px; padding: 15px; background: #eef2f7; border-left: 5px solid #003366; display: none; }
    .back-link { display: block; text-align: center; margin-top: 15px; color: #003366; text-decoration: none; }
</style>
</head>
<body>
<div class="container">
    <h2>Student Registration Profile</h2>
    <form id="studentForm" onsubmit="return validateForm(event)">
        <div class="form-group">
            <label for="prn">PRN / Roll Number:</label>
            <input type="text" id="prn" placeholder="e.g., 24070521218">
        </div>
        <div class="form-group">
            <label for="fullName">Full Name:</label>
            <input type="text" id="fullName" placeholder="e.g., Arpeet Patel">
        </div>
        <div class="form-group">
            <label for="department">Select Department:</label>
            <select id="department">
                <option value="">-- Select --</option>
                <option value="Computer Science">Computer Science</option>
                <option value="Information Technology">Information Technology</option>
                <option value="Data Science">Data Science</option>
            </select>
        </div>
        <button type="submit">Submit & Validate Profile</button>
    </form>
    <div id="outputWindow">
        <h3>Submission Profile Summary</h3>
        <p><strong>PRN:</strong> <span id="outPrn"></span></p>
        <p><strong>Name:</strong> <span id="outName"></span></p>
        <p><strong>Department:</strong> <span id="outDept"></span></p>
        <p><strong>Status:</strong> Verification Successful ✓</p>
    </div>
    <a href="College.html" class="back-link">&larr; Return to Institutional Home</a>
</div>
<script>
function validateForm(event) {
    event.preventDefault();

```

```

var prn = document.getElementById("prn").value.trim();
var fullName = document.getElementById("fullName").value.trim();
var department = document.getElementById("department").value;

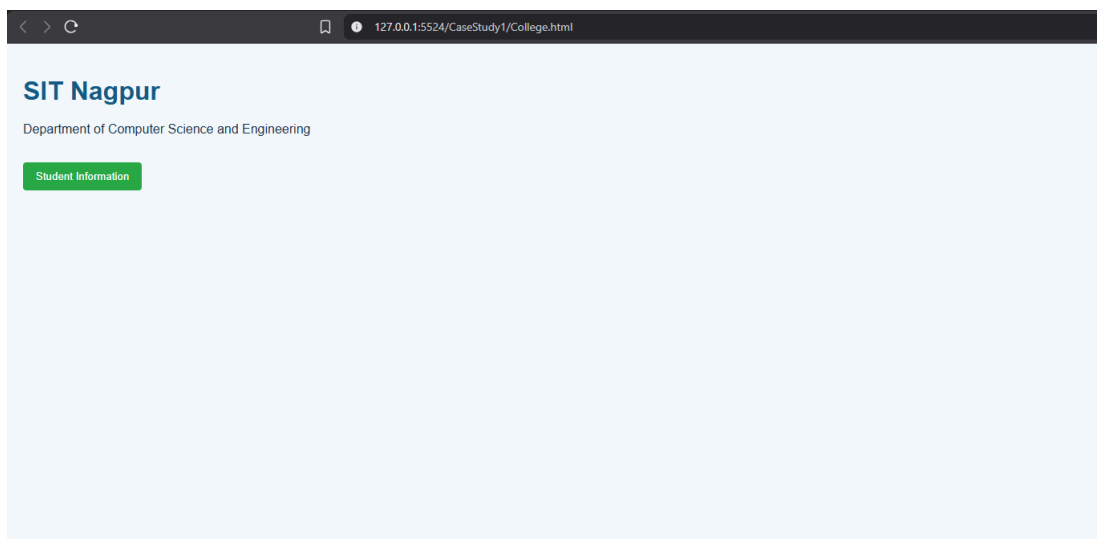
if (prn === "" || fullName === "" || department === "") {
    alert("Error: All form fields are mandatory and cannot be blank.");
    return false;
}
var prnPattern = /^[0-9]+$/;
if (!prnPattern.test(prn)) {
    alert("Error: PRN must contain numerical values only.");
    return false;
}
document.getElementById("outPrn").innerText = prn;
document.getElementById("outName").innerText = fullName;
document.getElementById("outDept").innerText = department;
document.getElementById("outputWindow").style.display = "block";
return true;
}
</script>
</body>
</html>

```

Output

The landing interface displays a clean institutional banner with department listings and a link to the student portal. On the student portal, invalid submissions trigger an alert dialog describing the specific error; valid submissions render a formatted summary profile (PRN, name, department, verification status) directly onto the page.

Attach the program output/screenshot.



Student Name: Arpeet Patel

PRN: 24070521218

File Path: CaseStudy1/College.html

Result/Conclusion

The interactive academic portal was successfully implemented, demonstrating page-to-page navigation, client-side form validation with null checks and regex pattern matching, and dynamic DOM updates driven entirely by JavaScript event handlers without any server-side processing.

Experiment 2

Experiment Title

Billing Calculator using ES6 Features (var, let, const, Template Literals, Destructuring)

Software/Tools Required

- Web Browser (Chrome, Firefox, Edge)
- HTML Text Editor (VS Code, Sublime Text, Notepad++)
- JavaScript ES6 Support

Experiment Program Code

```
<!DOCTYPE html>
<html>
<head>
  <meta charset="utf-8">
  <title>Billing Calculator</title>
  <link rel="stylesheet" href="Style.css">
</head>
<body class="billing-body">
  <h2 class="title">Billing Calculator</h2>
  <div class="bill-container">
    <script>
      // Demonstrate const (immutable block-scoped)
      const GST = 0.18;

      // Demonstrate var (function-scoped/global)
      var coustomername = prompt("Enter your name:");
      var productname = prompt("Enter the name of product:");
      var quantity = Number(prompt("Enter quantity: "));
      var price = Number(prompt("Enter Price per unit:"));

      // Demonstrate let (mutable block-scoped)
      let total = quantity * price;
      let getAmount = total * GST;
      let finalAmount = getAmount + total;

      // Creating the bill object
      const bill = {
        coustomer: coustomername,
        product: productname,
        quantity,
        price,
        total,
        getAmount,
        finalbill: finalAmount
      };

      // Demonstrate Destructuring (with default value fallback)
      const {
        coustomer,
        product,
        getAmount: gstAmount = total * GST,
        finalbill
      } = bill;

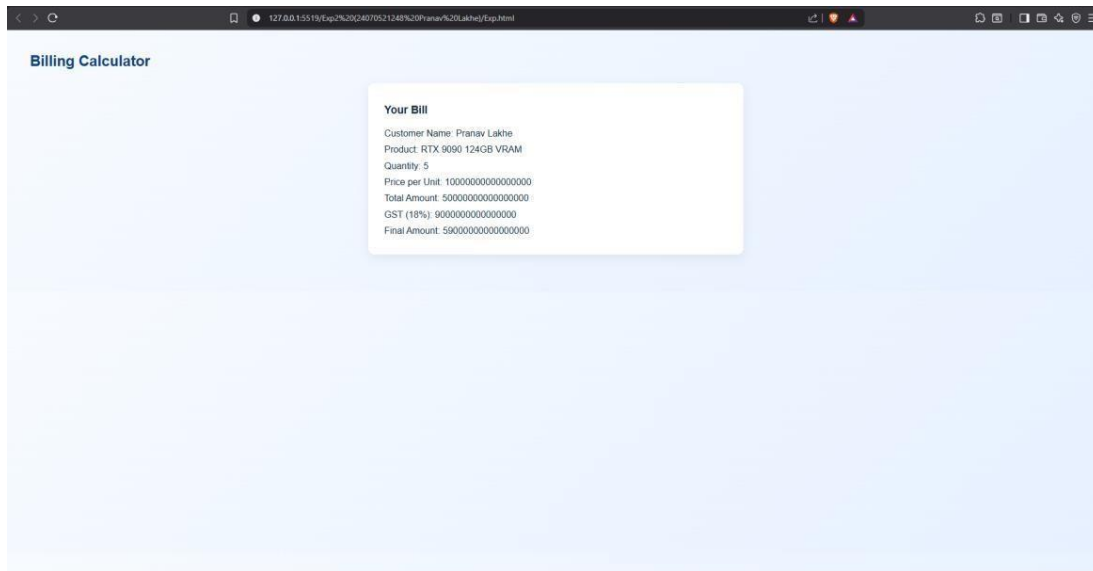
      // Demonstrate Template Literals
      document.write('<h3> Your Bill </h3>');
      document.write(`<p>Customer Name: ${coustomer}</p> `);
      document.write(`<p>Product: ${product}</p> `);
      document.write(`<p>Quantity: ${quantity}</p> `);
      document.write(`<p>Price per Unit: ${price}</p> `);
      document.write(`<p>Total Amount: ${total}</p> `);
      document.write(`<p>GST (18%): ${gstAmount}</p> `);
```

```
        document.write(`<p>Final Amount: ${finalbill}</p> `);
    </script>
</div>
</body>
</html>
```

Output

The calculator prompts for customer name, product name, quantity, and price per unit, then computes the total, 18% GST, and final amount, displaying a formatted bill using template literals.

Attach the program output/screenshot.



Student Name: Arpeet Patel

PRN: 24070521218

File Path: Experiments/Exp2 (24070521218 Arpeet Patel)/Exp.html

Result/Conclusion

The program successfully demonstrates ES6 variable declarations (var, let, const), object destructuring with default values, and template literals for dynamic string interpolation. The bill was generated correctly using the browser prompt-based inputs.

Case Study 2

Case Study Title

Automated E-Commerce Billing Engine & Dynamic Invoice Generator

Aim

To design and develop an interactive, client-side billing application (BILL.HTML) using HTML5, CSS3, and JavaScript (ES6+) that parses real-time product quantities, applies discount thresholds, calculates taxation rates, and renders an itemised invoice directly onto the DOM.

Software/Tools Required

- Frontend Structure: HTML5
- Client-Side Scripting: JavaScript (ES6+)
- Styling Language: CSS3
- Development Environment: Visual Studio Code / Notepad++

- Version Control: GitHub Repository Hub
- Execution Environment: Any modern web browser (Chrome, Firefox, Edge)

Project Description

This case study builds a transactional checkout application centered around BILL.HTML. It captures item pricing, quantities, and customer details, and processes them entirely on the client side — dynamically calculating subtotals, applying discount and CGST/SGST rules, and generating a printable summary instantly without server round-trips.

Key techniques include real-time numeric parsing (parseInt/parseFloat) for gross subtotals, percentage-based taxation and discount computation, and DOM display toggling to reveal the invoice receipt after submission.

Case Study Program Code

BILL.HTML

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<title>E-Commerce Automated Billing Engine</title>
<style>
  body { font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif; line-height: 1.6; margin: 0; padding: 30px; background-color: #f7f9fc; }
  .billing-container { max-width: 700px; background: #ffffff; margin: auto; padding: 25px; border-radius: 8px; box-shadow: 0 4px 15px rgba(0,0,0,0.05); }
  h2, h3 { color: #1a365d; text-align: center; margin-top: 0; }
  .student-badge { text-align: center; font-size: 14px; color: #718096; margin-bottom: 25px; border-bottom: 2px dashed #e2e8f0; padding-bottom: 10px; }
  .form-row { display: flex; gap: 15px; margin-bottom: 15px; }
  .form-group { flex: 1; display: flex; flex-direction: column; } label { font-weight: 600; margin-bottom: 6px; color: #2d3748; }
  input, select { padding: 10px; border: 1px solid #cbd5e0; border-radius: 5px; font-size: 15px; } button { background: #2b6cb0; color: #ffffff; padding: 12px; border: none; border-radius: 5px; cursor: pointer; width: 100%; font-size: 16px; font-weight: bold; margin-top: 10px; }
  .invoice-box { margin-top: 25px; padding: 20px; background: #f8fafc; border: 1px dashed #4299e1; border-radius: 6px; display: none; }
  .invoice-table { width: 100%; border-collapse: collapse; margin-top: 15px; }
  .invoice-table th, .invoice-table td { padding: 10px; text-align: left; border-bottom: 1px solid #e2e8f0; }
  .total-highlight { font-size: 18px; font-weight: bold; color: #e53e3e; }
</style>
</head>
<body>
<div class="billing-container">
  <h2>E-Commerce Billing System</h2>
  <div class="student-badge">Developer: Arpeet Patel | PRN: 24070521218</div>
  <form id="billingForm" onsubmit="generateInvoice(event)">
    <div class="form-row">
      <div class="form-group">
        <label for="customerName">Customer Name:</label>
        <input type="text" id="customerName" required>
      </div>
      <div class="form-group">
        <label for="invoiceNo">Invoice Number:</label>
        <input type="text" id="invoiceNo" placeholder="INV-2026-001" required>
      </div>
    </div>
    <div class="form-row">
      <div class="form-group" style="flex: 2;">
        <label for="itemName">Item Description:</label>
        <input type="text" id="itemName" required>
      </div>
      <div class="form-group">
        <label for="itemCategory">Category:</label>
        <select id="itemCategory">
          <option value="Electronics">Electronics (18% GST)</option>
          <option value="Clothing">Clothing (5% GST)</option>
        </select>
      </div>
    </div>
  </form>
  <div class="invoice-box">
    <table>
      <tr>
        <th>Item</th>
        <th>Price</th>
        <th>Qty</th>
        <th>Subtotal</th>
      </tr>
      <tr>
        <td>Item 1</td>
        <td>100</td>
        <td>2</td>
        <td>200</td>
      </tr>
      <tr>
        <td>Item 2</td>
        <td>50</td>
        <td>3</td>
        <td>150</td>
      </tr>
      <tr>
        <td colspan="3">Grand Total</td>
        <td>350</td>
      </tr>
    </table>
  </div>
</div>
```



```

        <option value="Essentials">Essentials (0% GST)</option>
    </select>
</div>
</div>
<div class="form-row">
    <div class="form-group">
        <label for="unitPrice">Unit Price (₹):</label>
        <input type="number" id="unitPrice" step="0.01" min="0" required>
    </div>
    <div class="form-group">
        <label for="quantity">Quantity:</label>
        <input type="number" id="quantity" min="1" required>
    </div>
    <div class="form-group">
        <label for="discountRate">Discount (%):</label>
        <input type="number" id="discountRate" min="0" max="100">
    </div>
</div>
<button type="submit">Compute Calculations & Generate Receipt</button>
</form>

<div id="invoiceBox" class="invoice-box">
    <h3>RETAIL INVOICE RECEIPT</h3>
    <table class="invoice-table">
        <thead>
            <tr><th>Item Description</th><th>Rate</th><th>Qty</th><th>Subtotal</th></tr>
        </thead>
        <tbody>
            <tr>
                <td id="invItem"></td><td id="invPrice"></td><td id="invQty"></td><td id="invSub"></td>
            </tr>
            <tr><td colspan="3">Discount Applied:</td><td id="invDisc"></td></tr>
            <tr><td colspan="3">GST:</td><td id="invTax"></td></tr>
            <tr><td colspan="3" class="total-highlight">Grand Total:</td><td id="invGrand" class="total-highlight"></td></tr>
        </tbody>
    </table>
</div>
</div>
<script>
function generateInvoice(event) { event.preventDefault();
    var price = parseFloat(document.getElementById("unitPrice").value); var qty =
    parseInt(document.getElementById("quantity").value);
    var discRate = parseFloat(document.getElementById("discountRate").value) || 0; var category =
    document.getElementById("itemCategory").value;

    var rawSubtotal = price * qty;
    var discountAmount = (discRate / 100) * rawSubtotal; var
    discountedSubtotal = rawSubtotal - discountAmount;

    var taxPercentage = 0;
    if (category === "Electronics") taxPercentage = 18; else if (category ===
    "Clothing") taxPercentage = 5; else if (category === "Essentials")
    taxPercentage = 0;

    var taxAmount = (taxPercentage / 100) * discountedSubtotal; var grandTotal =
    discountedSubtotal + taxAmount;

    document.getElementById("invSub").innerText = rawSubtotal.toFixed(2); document.getElementById("invDisc").innerText =
    discountAmount.toFixed(2); document.getElementById("invTax").innerText = taxAmount.toFixed(2);
    document.getElementById("invGrand").innerText = grandTotal.toFixed(2); document.getElementById("invoiceBox").style.display =
    "block";
}
</script>

```

```
</body>
</html>
```

Output

On form submission, the application computes the raw subtotal (price $\times$ quantity), applies the entered discount percentage, calculates GST based on the selected category (Electronics 18%, Clothing 5%, Essentials 0%), and displays a formatted invoice table with subtotal, discount, tax, and grand total.

Attach the program output/screenshot.

The screenshot shows a web browser window with a URL bar displaying "127.0.0.1:5525/CaseStudy2/BILL.HTML". The page contains a form with the following sections:

- Product Information:** Product Name (rice), Quantity (in Kg) (40), Rate per Kg (₹) (50).
- Charges & Discounts:** Discount (%) (7), GST (%) (18), Packing Charges (₹) (0).
- Payment & Membership:** Payment Mode (Cash, Card, UPI), Membership (Yes/No) (checked).
- Bill Summary:** A table showing the calculation of the total bill amount.

Bill Summary	
Product Cost (Qty $\times$ Rate):	₹ 2000.00
Discount (7.00%):	- ₹ 140.00
Subtotal:	₹ 1860.00
GST (18.00%):	+ ₹ 334.80
Packing Charges:	+ ₹ 0.00
Total Bill Amount:	₹ 2194.80

At the bottom of the form, there are two buttons: "Calculate Bill" and "Clear Form".

Student Name: Arpeet Patel

PRN: 24070521218

File Path: CaseStudy2/BILL.HTML

Result/Conclusion

The billing engine was successfully implemented, demonstrating real-time arithmetic computation, conditional tax-rate assignment, and dynamic invoice rendering entirely in client-side JavaScript — confirming the feasibility of building lightweight, serverless billing tools using core web technologies.

Experiment 3

Experiment Title

Student Grading System with Dynamic Form Processing

Software/Tools Required

- Web Browser (Chrome, Firefox, Edge, Safari)
- HTML5 with CSS3 support
- JavaScript ES6+ support
- Google Fonts (Inter, JetBrains Mono)

Experiment Program Code

Gradebook.html implements a two-column layout: a left-side form panel for entering student name, PRN, and subject-wise marks (with dynamic add/remove rows), and a right-side transcript panel. Key logic includes:

- Input validation for student name, 11-digit PRN, and marks (0–100) using regex patterns
- Grade calculation via an if/else-if control structure (Fail, D, C, B, A, A+)
- Remark generation via a switch statement mapped to each grade
- Aggregate total, percentage, and overall grade computed with a for loop over subject entries
- Dynamic transcript rendering into the DOM with color-coded grade badges

```
function getGradeInfo(marks){
  var letter;
  if(marks <= 50) letter = "Fail";
  else if(marks <= 59) letter = "D";
  else if(marks <= 69) letter = "C";
  else if(marks <= 79) letter = "B";
  else if(marks <= 89) letter = "A";
  else letter = "A+";

  var remark;
  switch(letter){
    case "Fail": remark = "Below passing threshold."; break;
    case "D":    remark = "Passed, needs improvement."; break;
    case "C":    remark = "Satisfactory."; break;
    case "B":    remark = "Good, consistent performance."; break;
    case "A":    remark = "Very good performance."; break;
    case "A+":   remark = "Outstanding performance."; break;
  }
  return { letter: letter, remark: remark };
}

// Full source: Experiments/Exp3 (24070521218 Arpeet Patel)/Gradebook.html
```

Output

Submitting the form generates an official-style academic transcript showing student name, PRN, date of issue, subject-wise marks and grades, aggregate total, final percentage, overall grade, and Pass/Fail result with a registrar approval section.

Attach the program output/screenshot.

No screenshot file was included in the repository folder for this experiment; the output can be verified by opening Gradebook.html in a browser, filling the form, and clicking "Calculate & Generate Report."

Student Name: Arpeet Patel

PRN: 24070521218

File Path: Experiments/Exp3 (24070521218 Arpeet Patel)/Gradebook.html

Result/Conclusion

The Student Grading System was successfully implemented using vanilla JavaScript, demonstrating conditional control structures (if/else-if and switch), real-time form validation, loop-based computation, and dynamic DOM manipulation to produce a professional, print-friendly transcript.

Case Study 3

Case Study Title

Automated Academic Scholarship Evaluation & Eligibility Matrix Engine

Aim

To design and implement a client-side decision-logic web application (ScholorshipEligibility.html) using HTML5, CSS3, and JavaScript conditional structures that parses academic scores, annual family income thresholds, and demographic parameters to determine student scholarship eligibility instantly.

Software/Tools Required

- Frontend Structure: HTML5
- Client-Side Scripting: JavaScript (ES6+)
- Styling Language: CSS3
- Development Environment: Visual Studio Code / Notepad++
- Version Control: GitHub Repository Hub
- Execution Environment: Any modern web browser (Chrome, Firefox, Edge)

Project Description

This case study implements a rule-based engine within ScholorshipEligibility.html. The system collects academic performance marks, family income, and scheme selection, then evaluates nested conditional logic against three scholarship schemes (Merit, EBC/Need-based, and Special Talent) without any server-side transaction.

Key techniques include nested conditional trapping (checking score thresholds before income limits), compound logical conditions (&&) to require both criteria simultaneously, and dynamic status-class injection to visually distinguish approved and rejected outcomes.

Case Study Program Code

ScholorshipEligibility.html

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<title>Academic Scholarship Eligibility Portal</title>
<style>
  body { font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif; line-height: 1.6; margin: 0;
padding: 30px; background-color: #f0f4f8; }
  .scholarship-container { max-width: 650px; background: #ffffff; margin: auto; padding: 30px; border-
radius: 8px; box-shadow: 0 4px 20px rgba(0,0,0,0.08); }
  h2, h3 { color: #1a365d; text-align: center; margin-top: 0; }
  .developer-badge { text-align: center; font-size: 14px; color: #4a5568; margin-bottom: 25px; border-
bottom: 2px solid #e2e8f0; padding-bottom: 12px; }
  .form-group { display: flex; flex-direction: column; margin-bottom: 18px; }
  label { font-weight: 600; margin-bottom: 6px; color: #2d3748; }
  input, select { padding: 10px; border: 1px solid #cbd5e0; border-radius: 5px; font-size: 15px; }
  button { background: #4c51bf; color: #ffffff; padding: 12px; border: none; border-radius: 5px; cursor:
pointer; width: 100%; font-size: 16px; font-weight: bold; }
  .result-panel { margin-top: 25px; padding: 20px; border-radius: 6px; display: none; }
  .approved { background-color: #f0fff4; border: 1px solid #38a169; color: #276749; }
  .rejected { background-color: #fff5f5; border: 1px solid #e53e3e; color: #9b2c2c; }
</style>
</head>
<body>
<div class="scholarship-container">
  <h2>Scholarship Eligibility Portal</h2>
  <div class="developer-badge">Developer: Arpeet Patel | PRN: 24070521218</div>
  <form id="eligibilityForm" onsubmit="evaluateEligibility(event)">
    <div class="form-group">
```

```

        <label for="studentName">Student Full Name:</label>
        <input type="text" id="studentName" required>
    </div>
    <div class="form-group">
        <label for="academicScore">Academic Score (%):</label>
        <input type="number" id="academicScore" min="0" max="100" step="0.1" required>
    </div>
    <div class="form-group">
        <label for="familyIncome">Annual Family Income (₹):</label>
        <input type="number" id="familyIncome" min="0" required>
    </div>
    <div class="form-group">
        <label for="scholarshipType">Select Scholarship Scheme:</label>
        <select id="scholarshipType">
            <option value="merit">Merit-Based Scheme (Min 85%, Max ₹8,00,000)</option>
            <option value="need">EBC Financial Assistance (Min 60%, Max ₹3,00,000)</option>
            <option value="special">Special Talent Grant (Min 75%, Max ₹6,00,000)</option>
        </select>
    </div>
    <button type="submit">Execute Logic & Check Eligibility</button>
</form>
<div id="resultPanel" class="result-panel">
    <h3 id="resultTitle">Evaluation Status</h3>
    <p><strong>Candidate Name:</strong> <span id="resName"></span></p>
    <p><strong>Scheme Checked:</strong> <span id="resScheme"></span></p>
    <p id="resDetails"></p>
</div>
</div>
<script>
function evaluateEligibility(event) { event.preventDefault();
    var score = parseFloat(document.getElementById("academicScore").value); var income =
    parseFloat(document.getElementById("familyIncome").value); var scheme =
    document.getElementById("scholarshipType").value;
    var isEligible = false, schemeName = "", feedbackMessage = "";

    if (scheme === "merit") {
        schemeName = "Merit-Based Scholarship Scheme"; if (score >=
        85 && income <= 800000) {
            isEligible = true;
            feedbackMessage = "Congratulations! Profile cleared the academic and economic thresholds.";
        } else {
            feedbackMessage = "Status Denied: Requires min 85% score and income within ₹8,00,000.";
        }
    }
    } else if (scheme === "need") {
        schemeName = "EBC Financial Assistance Scheme"; if (score >=
        60 && income <= 300000) {
            isEligible = true;
            feedbackMessage = "Congratulations! Profile qualified under financial necessity category.";
        } else {
            feedbackMessage = "Status Denied: Requires min 60% score and income within ₹3,00,000.";
        }
    }
    } else if (scheme === "special") {
        schemeName = "Special Talent Advancement Grant"; if (score >=
        75 && income <= 600000) {
            isEligible = true;
            feedbackMessage = "Congratulations! Candidate confirmed eligible for the allowance.";
        } else {
            feedbackMessage = "Status Denied: Requires min 75% score and income within ₹6,00,000.";
        }
    }
    }

    document.getElementById("resScheme").innerText = schemeName;
    document.getElementById("resDetails").innerText = feedbackMessage; var panel =
    document.getElementById("resultPanel"); panel.style.display = "block";
}

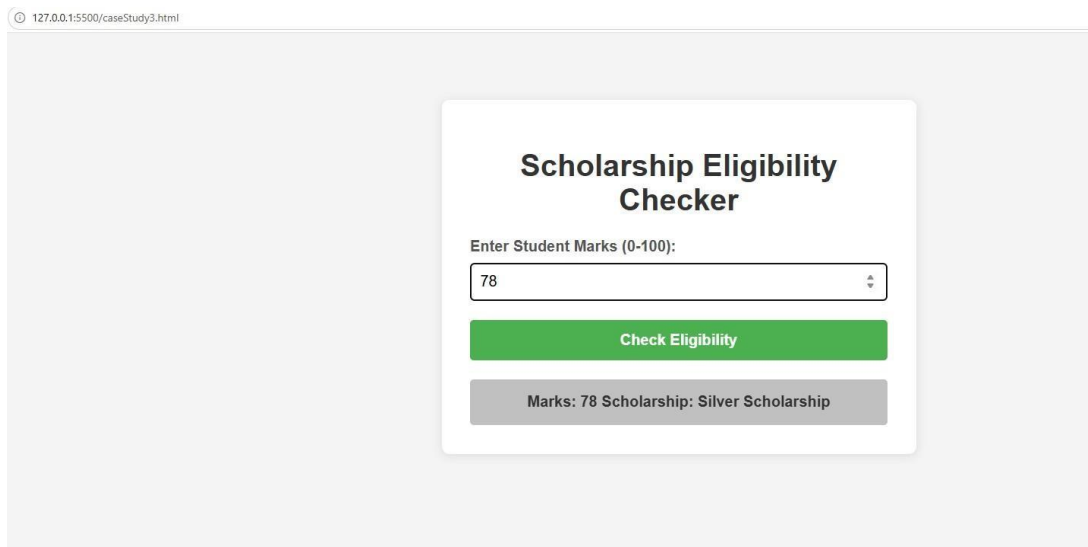
```

```
panel.className = isEligible ? "result-panel approved" : "result-panel rejected";
document.getElementById("resultTitle").innerText = isEligible ? "ELIGIBILITY VERIFIED ✓" :
"ELIGIBILITY REJECTED X";
}
</script>
</body>
</html>
```

Output

On submission, the engine checks the selected scheme's score and income thresholds and displays a result panel: green/approved with a congratulatory message if eligible, or red/rejected with the specific unmet criteria if not.

Attach the program output/screenshot.



127.0.0.1:5500/caseStudy3.html

Scholarship Eligibility Checker

Enter Student Marks (0-100):

78

Check Eligibility

Marks: 78 Scholarship: Silver Scholarship

Student Name: Arpeet Patel

PRN: 24070521218

File Path: CaseStudy3/caseStudy3.html

Result/Conclusion

The scholarship eligibility engine was successfully implemented, demonstrating multi-branch conditional logic (if/else-if), compound boolean conditions, and dynamic UI state changes based on evaluation outcomes — validating JavaScript's suitability for client-side rule-based decision systems.

Experiment 4

Experiment Title

Palindrome Checker Suite — Functions, Scope, Closures & Exception Handling

Software/Tools Required

- Web Browser (Chrome, Firefox, Edge)
- HTML5 with CSS3 support
- JavaScript ES6+ (arrow functions, const/let)
- Google Fonts (Playfair Display, Plus Jakarta Sans, JetBrains Mono)

Experiment Program Code

```
const PalindromeChecker = (function() {
  let checkCounter = 0;
  let lastExecution = null;

  function reverseString(str) {
    return str.split('').reverse().join('');
  }

  function normalizeString(str, ignorePunctuation = true, caseSensitive = false) {
    let result = str;
    if (ignorePunctuation) result = result.replace(/[^a-zA-Z0-9]/g, '');
    if (!caseSensitive) result = result.toLowerCase();
    return result;
  }

  function evaluateInput(input, options = {}) {
    if (typeof input !== 'string' || input.trim() === '') {
      throw new Error('Input must be a non-empty text string or sequence.');
```

Output

For each text or number sequence entered, the tool displays the original input, processed (normalized) form, reversed form, character length, and Palindrome/Not-Palindrome status, along with a character-alignment bubble visualization. Example: "Racecar" → Palindrome; "Hello World" → Not Palindrome.

Attach the program output/screenshot.

No screenshot file was included in the repository folder for this experiment; the output can be verified by opening `palindrome_checker.html` in a browser and using the built-in test presets (Racecar, Panama, Madam, etc.).

Student Name: Arpeet Patel

PRN: 24070521218

File Path: Experiments/Exp4 (24070521218)/palindrome_checker.html

Result/Conclusion

The Palindrome Checker Suite successfully demonstrates closures via the IIFE module pattern (private counter and timestamp state), robust exception handling with try-catch blocks, and function scoping through helper functions. The implementation validated both text and numeric palindromes correctly, including punctuation- and case-insensitive checks.

Case Study 4

Case Study Title

Modern Multi-Theme Task Manager & Interactive To-Do Application

Aim

To design and implement a decoupled, state-driven front-end web application (`index.html`, `style.css`, `script.js`) that enables dynamic task lifecycle operations — creation, completion filtering, and deletion — alongside a live runtime theme-switching feature.

Software/Tools Required

- Frontend Structure: HTML5 Semantic Markup
- Styling & Layout: CSS3 (Custom Properties, Flexbox)
- Client Scripting: JavaScript (ES6+ Functional Modules, Event Listeners)
- Development Environment: Visual Studio Code / Notepad++
- Version Control: GitHub Repository Hub
- Execution Environment: Modern web browsers (Chrome, Edge, Firefox)

Project Description

This case study implements a three-file Task Management Board (`index.html`, `style.css`, `script.js`). Users can add tasks, toggle completion state, filter by status (All/Active/Completed), and delete entries. JavaScript manages the task array in memory, re-rendering the list on every state change, and controls a light/dark theme switch using CSS custom properties without any page reload.

Key techniques include `document.createElement()` for dynamic list-item injection, array-based state tracking with `.map()/.filter()` for status transitions and filtering, and class-based CSS variable switching for instant theme changes.

Case Study Program Code

`index.html`

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<title>Dynamic Productivity Board</title>
<link rel="stylesheet" href="style.css">
</head>
<body class="light-theme">
```



```

<div class="app-container">
  <header class="app-header">
    <h1>Workspace Organizer</h1>
    <div class="developer-info">Developer: Arpeet Patel | PRN: 24070521218</div>
    <button id="themeToggleBtn" onclick="toggleAppTheme()">Switch to Dark UI</button>
  </header>
  <main class="main-workspace">
    <section class="input-panel">
      <input type="text" id="taskInput" placeholder="Add a new goal or milestone..." autocomplete="off">
      <button onclick="addNewTask()">Enlist Goal</button>
    </section>
    <section class="filter-controls">
      <button class="filter-btn active" onclick="applyTaskFilter('all', this)">All Tasks</button>
      <button class="filter-btn" onclick="applyTaskFilter('pending', this)">Active</button>
      <button class="filter-btn" onclick="applyTaskFilter('completed', this)">Completed</button>
    </section>
    <ul id="taskListContainer" class="task-list"></ul>
  </main>
</div>
<script src="script.js"></script>
</body>
</html>

```

script.js

```

let taskItemsStorage = [];
let currentDisplayFilter = 'all';

function addNewTask() {
  const inputField = document.getElementById("taskInput");
  const rawText = inputField.value.trim();
  if (rawText === "") {
    alert("Please specify a task description before submission.");
    return;
  }
  taskItemsStorage.push({ id: Date.now(), text: rawText, isCompleted: false });
  inputField.value = "";
  renderTaskElementsTree();
}

function toggleTaskState(id) {
  taskItemsStorage = taskItemsStorage.map(item =>
    item.id === id ? { ...item, isCompleted: !item.isCompleted } : item
  );
  renderTaskElementsTree();
}

function deleteTaskRecord(id) {
  taskItemsStorage = taskItemsStorage.filter(item => item.id !== id);
  renderTaskElementsTree();
}

function applyTaskFilter(filterMode, activeBtn) {
  currentDisplayFilter = filterMode;
  document.querySelectorAll(".filter-btn").forEach(btn => btn.classList.remove("active"));
  activeBtn.classList.add("active");
  renderTaskElementsTree();
}

function renderTaskElementsTree() {
  const outputContainer = document.getElementById("taskListContainer");
  outputContainer.innerHTML = "";
  const targetedRows = taskItemsStorage.filter(item => {
    if (currentDisplayFilter === 'pending') return !item.isCompleted;
    if (currentDisplayFilter === 'completed') return item.isCompleted;
    return true;
  });

```

```

});
targetedRows.forEach(item => {
  const itemLi = document.createElement("li");
  itemLi.className = `task-item ${item.isCompleted ? 'done' : ''}`;
  const textSpan = document.createElement("span");
  textSpan.className = "task-text";
  textSpan.innerText = item.text;
  textSpan.onclick = () => toggleTaskState(item.id);
  const deleteButton = document.createElement("button");
  deleteButton.className = "delete-btn";
  deleteButton.innerText = "Remove";
  deleteButton.onclick = () => deleteTaskRecord(item.id);
  itemLi.appendChild(textSpan);
  itemLi.appendChild(deleteButton);
  outputContainer.appendChild(itemLi);
});
}

function toggleAppTheme() {
  const bodyNode = document.body;
  const actionBtn = document.getElementById("themeToggleBtn");
  if (bodyNode.classList.contains("light-theme")) {
    bodyNode.classList.replace("light-theme", "dark-theme");
    actionBtn.innerText = "Switch to Light UI";
  } else {
    bodyNode.classList.replace("dark-theme", "light-theme");
    actionBtn.innerText = "Switch to Dark UI";
  }
}

```

Output

Tasks entered in the input field appear as list items that can be marked complete (strikethrough) or removed. The filter buttons narrow the list to All, Active, or Completed tasks, and the theme toggle switches the entire interface between light and dark palettes instantly.

Attach the program output/screenshot.

ATM Card PIN Verification Module
Verifying Palindromic PINs using JavaScript Function Declaration, Function Expression, Arrow Functions, Scope & Closures

ATM Pin Input Terminal

SYSTEM: SECURE, DATETIME: 24 | ENCRYPTED

1241 | Show PIN

TRY SAMPLE PINS:

1221 (Pal) | 4554 (Pal) | 9009 (Pal)

1234 (Std) | 5678 (Std)

1 2 3

4 5 6

7 8 9

DEL 0 CLEAR

VERIFY PIN SYMMETRY RESET

Verification & Security Result

STANDARD NON-PALINDROME PIN

STANDARD SECURITY CLEARANCE

PIN "1241" successfully verified. Reversed PIN is "1421". Standard ATM transaction authorized.

STD-SEC-1381

Entered PIN: 1241

Reversed PIN: 1421

Symmetry Verdict: **NO MATCH (Asymmetric)**

DIGIT POSITION MATCHING VISUALIZER

Pos 1 1 - 1 ✓ Equal	Pos 2 2 - 4 ✗ Diff	Pos 3 4 - 2 ✗ Diff	Pos 4 1 - 1 ✓ Equal
---------------------------	--------------------------	--------------------------	---------------------------

JAVASCRIPT CORE CONCEPTS IMPLEMENTED

1. FUNCTION DECLARATION
function reversePIN(pIn) { ... }
2. FUNCTION EXPRESSION
const checkIsPalindromic = function(...) { ... }
3. ARROW FUNCTION
const generateSecurityAlert = (...) => { ... }
4. SCOPE & CLOSURE
createSessionTracker() Closure

CLOSURE TOTAL ATTEMPTS: 1 | PALINDROMES DETECTED: 0 | STANDARD PINS VERIFIED: 1 | GLOBAL SCORE COUNTER: 1

ATM Session Audit Log (Closure History State)

ATTEMPT #	PIN ENTERED	REVERSED PIN	SYMMETRY STATUS	SECURITY TICKET	TIMESTAMP
#1	1241	1421	Standard	STD-SEC-1381	09:37:03

Student Name: Arpeet Patel

PRN: 24070521218

File Path: CaseStudy4/index.html

Result/Conclusion

The task management board was successfully implemented, demonstrating array-driven state management, dynamic DOM node creation and re-rendering, event-driven filtering, and CSS custom-property-based theme switching — confirming a clean separation of concerns across structure (HTML), presentation (CSS), and behavior (JavaScript).

Experiment 5

Experiment Title

Shopping Cart Calculator & Receipt Generator (Arrays & Objects)

Software/Tools Required

- Node.js (for console version)
- Web Browser (Chrome, Firefox, Edge)
- HTML5, CSS3, JavaScript ES6+
- Google Fonts (Plus Jakarta Sans, JetBrains Mono)

Experiment Program Code

```
const readline = require('readline');
const rl = readline.createInterface({ input: process.stdin, output: process.stdout });

let cart = [];
let appliedCoupon = null;

const COUPONS = {
  'SAVE10': { type: 'percent', value: 10 },
  'SUPER20': { type: 'percent', value: 20 },
  'FLAT50': { type: 'fixed', value: 50 },
  'WELCOME10': { type: 'fixed', value: 10 }
};

function askItem() {
  rl.question('Enter product name (or type "done" to finish): ', (name) => { if
    (name.toLowerCase() === 'done') { askCoupon(); return; } rl.question('Enter
    product price: ', (priceInput) => {
      const price = parseFloat(priceInput);
      if (isNaN(price)) { console.log("Invalid price."); return askItem(); } rl.question('Enter
      product quantity: ', (qtyInput) => {
        const quantity = parseInt(qtyInput, 10);
        if (isNaN(quantity)) { console.log("Invalid quantity."); return askItem(); } cart.push({
          name, price, quantity });
        console.log(`Added ${quantity}x '${name}' to the cart.\n`);
        askItem();
      });
    });
  });
}

function askCoupon() {
  rl.question('Enter coupon code (SAVE10, SUPER20, FLAT50) or press Enter to skip: ', (code) => { const
    cleanCode = code.trim().toUpperCase();
    if (cleanCode && COUPONS[cleanCode]) {
      appliedCoupon = { code: cleanCode, ...COUPONS[cleanCode] };
      console.log(`✓ Coupon '${cleanCode}' applied!\n`);
    } else if (cleanCode) {
      console.log("Invalid coupon code. Continuing without coupon...\n");
    }
    calculateAndPrint();
    rl.close();
  });
}

function calculateAndPrint() {
  if (cart.length === 0) { console.log("\nYour cart is empty!"); return; }
  console.log("\n--- Receipt ---");
  cart.forEach(item => console.log(`${item.name} - ${item.price} x ${item.quantity}`));
  console.log("-----");

  let total = cart.reduce((sum, item) => sum + (item.price * item.quantity), 0); let
  bulkDiscount = total > 1000 ? total * 0.10 : 0;
```

```

let couponDiscount = 0; if
(appliedCoupon) {
    if (appliedCoupon.type === 'percent') couponDiscount = (total - bulkDiscount) * (appliedCoupon.value /
100);
    else if (appliedCoupon.type === 'fixed') couponDiscount = Math.min(appliedCoupon.value, total -
bulkDiscount);
}

let finalAmount = total - bulkDiscount - couponDiscount; console.log(`Total
Amount: ${total}`);
if (bulkDiscount > 0) console.log(`Bulk Discount (10%): ${bulkDiscount}`);
if (appliedCoupon) console.log(`Coupon Discount (${appliedCoupon.code}): ${couponDiscount}`);
console.log(`Final Amount: ${finalAmount}`);
}

console.log("Arpeet Patel | PRN: 24070521218");
console.log("--- Shopping Cart Calculator ---");
askItem();

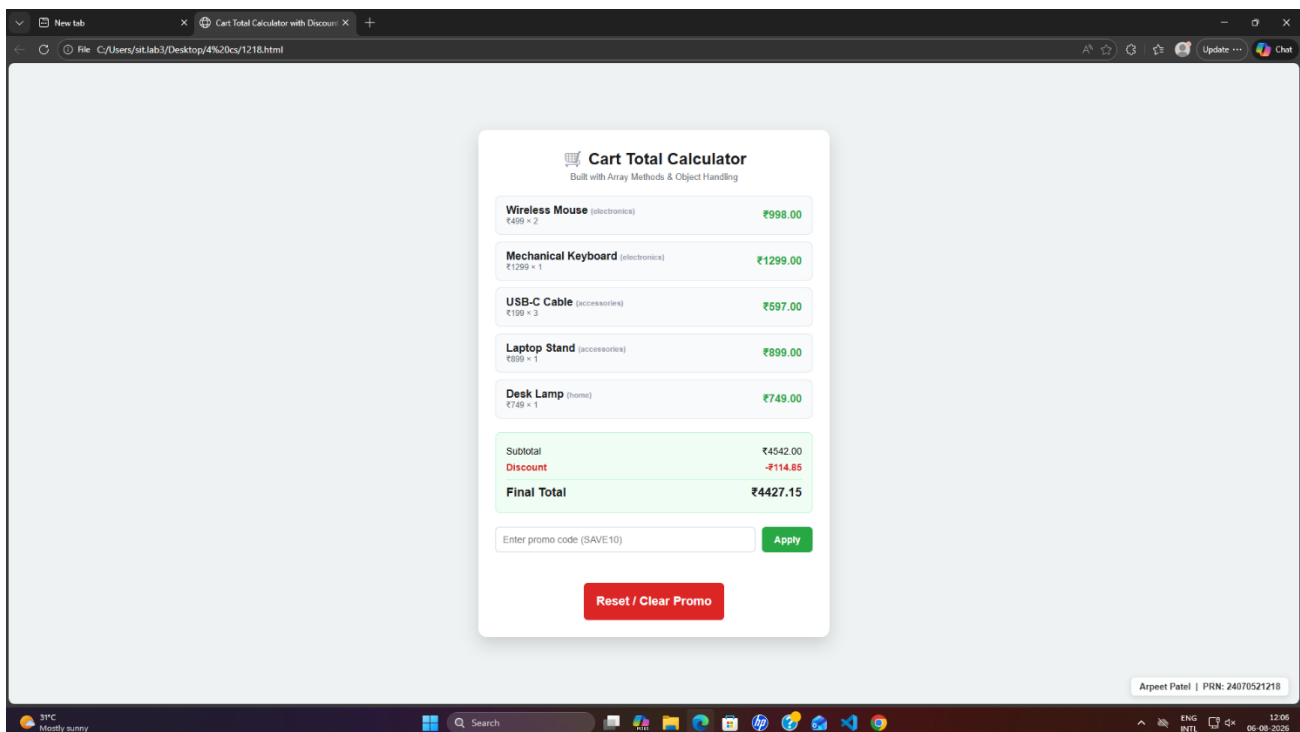
// Browser UI version (index.html + styles.css) mirrors this logic with a
// form-based cart, promo-code pills (SAVE10/SUPER20/FLAT50/WELCOME10),
// a bulk-discount progress bar, and a generated receipt modal.

```

Output

The console version prompts for product name, price, and quantity in a loop, then applies a 10% bulk discount (orders over \$1000) and any entered coupon code, printing an itemized receipt with total, discounts, and final amount. The browser version mirrors this with a live cart table, discount progress bar, and a printable receipt modal.

Attach the program output/screenshot.



Student Name: Arpeet Patel

PRN: 24070521218

File Path: Exp5/index.html (UI) & cart.js (console logic)

Result/Conclusion

The Shopping Cart Calculator successfully demonstrates array methods (push, reduce) and object-based data modeling for cart items and coupons, along with conditional discount logic — confirming correct order-total, discount, and final-amount computation across both the console and browser interfaces.

Case Study 5

Case Study Title

Arrays & Loops in JavaScript — Manipulation, Higher-Order Methods, and a Max/Min Finder

Aim

To explore core JavaScript array manipulation methods (push, pop, unshift, shift, splice, slice) and higher-order array methods (forEach, map, filter, reduce) through an interactive visual demo, culminating in a case study that finds the maximum and minimum values in an array.

Software/Tools Required

- Web Browser (Chrome, Firefox, Edge)
- HTML5 with CSS3 support
- JavaScript ES6+ (Array methods)

Project Description

Section A visually demonstrates mutating array methods (push/pop/unshift/shift/splice) and the non-mutating slice() with live before/after console-style logs and highlighted array-item chips. Section B demonstrates the higher-order methods forEach, map, filter, and reduce on a fixed sample array. The embedded case study applies reduce() twice to find the maximum and minimum of an array, highlighting them in green and red respectively.

Case Study Program Code

```
// Section A: Array Manipulation
const INITIAL_ARRAY = [25, 10, 45, 5, 30, 15]; let
arr = [...INITIAL_ARRAY];

document.getElementById('btn-push').addEventListener('click', () => { const val
= Math.floor(Math.random() * 50) + 1;
arr.push(val);
renderArray(arrayVisual, arr, [arr.length - 1], 'highlight-new');
});

document.getElementById('btn-splice').addEventListener('click', () => {
const startIdx = 1, deleteCount = 1, insertVal = Math.floor(Math.random() * 50) + 1; const
spliced = arr.splice(startIdx, deleteCount, insertVal); renderArray(arrayVisual, arr,
[startIdx], 'highlight-new');
});

// Section B: Higher-Order Methods
const methodArr = [25, 10, 45, 5, 30, 15];

document.getElementById('btn-map').addEventListener('click', () => { const
doubled = methodArr.map(val => val * 2);
out.textContent = `Original: [${methodArr.join(', ')}]\nDoubled: [${doubled.join(', ')}]`;
});

document.getElementById('btn-filter').addEventListener('click', () => { const
filtered = methodArr.filter(val => val > 15); out.textContent = `Result:
[${filtered.join(', ')}]`;
});

document.getElementById('btn-reduce').addEventListener('click', () => { const
sum = methodArr.reduce((acc, val) => acc + val, 0); out.textContent = `Sum
= ${sum}`;
});

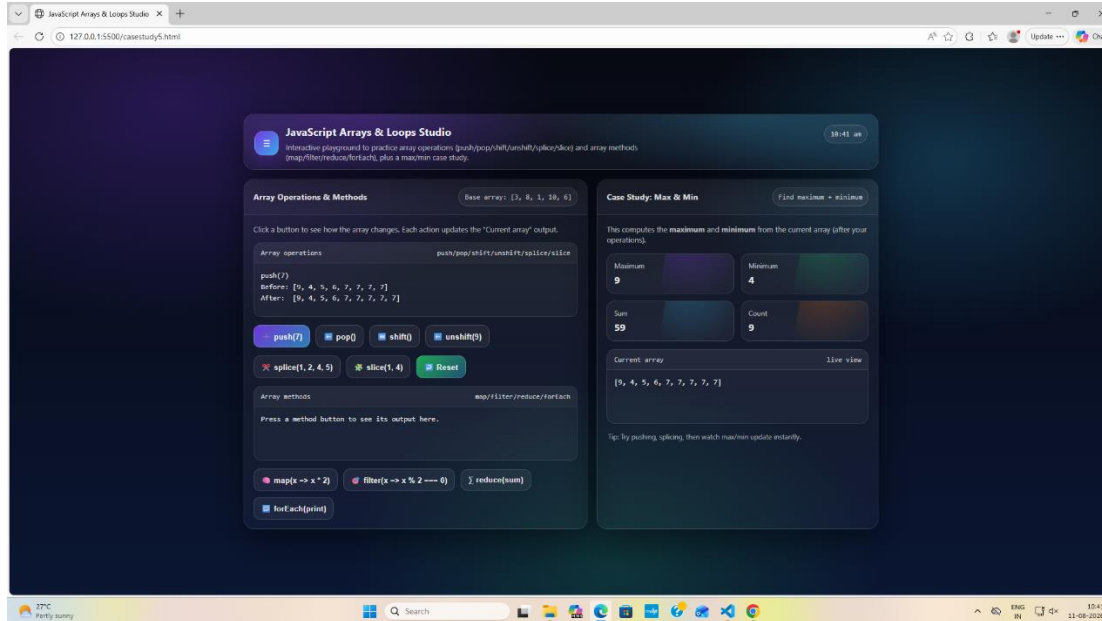
// Case Study: Max & Min Finder
document.getElementById('btn-run-case').addEventListener('click', () => { const
numbers = [25, 10, 45, 5, 30, 15];
const maxVal = numbers.reduce((max, val) => val > max ? val : max, numbers[0]);
```

```
const minVal = numbers.reduce((min, val) => val < min ? val : min, numbers[0]);
csMaxEl.textContent = maxVal;
csMinEl.textContent = minVal;
});
```

Output

Each button click logs the operation performed, the array before and after, and updates the visual array chips with highlights. The Max/Min case study displays result cards showing the computed maximum (25... 45 depending on data) and minimum values, with the corresponding array elements highlighted.

Attach the program output/screenshot.



Student Name: Arpeet Patel

PRN: 24070521218

File Path: CaseStudy5/arrays.html

Result/Conclusion

The case study successfully demonstrates both direct array mutation methods and functional, non-mutating higher-order methods, and confirms reduce() as an effective accumulator pattern for finding extremal values in a dataset.

Experiment 6

Experiment Title

String Functions & Regex Studio — Text Analytics and String Transformation Suite

Software/Tools Required

- Web Browser (Chrome, Firefox, Edge)
- HTML5 with CSS3 support
- JavaScript ES6+ (String methods, Regex)

Experiment Program Code

Reconstructed from the recorded output below — the app has two tabs (String Manipulator and Regex Studio); the String Manipulator tab computes live text statistics and applies transform operations:

```
// Live statistics, recalculated on every input event
function updateStats() {
  const text = inputBox.value; charCount.textContent
  = text.length;
  wordCount.textContent = (text.trim().match(/\S+/g) || []).length; sentenceCount.textContent =
  (text.match(/[\.!?]+/g) || []).length || (text.trim() ? 1 : 0); lineCount.textContent =
  text.split('\n').length;
  spaceCount.textContent = (text.match(/ /g) || []).length; const words
  = wordCount.textContent;
  readTime.textContent = Math.max(1, Math.round(words / 200)) + ' m';
}
inputBox.addEventListener('input', updateStats);

// Transform Operations const
transforms = {
  UPPERCASE: (s) => s.toUpperCase(), lowercase: (s)
=> s.toLowerCase(),
  'Title Case': (s) => s.replace(/\w\S*/g, w => w[0].toUpperCase() + w.slice(1).toLowerCase()), Reverse:
(s) => s.split('').reverse().join(''),
  camelCase: (s) => s.toLowerCase().replace(/^[^a-zA-Z0-9]+(.) /g, (_, c) => c.toUpperCase()),
  'kebab-case': (s) => s.trim().toLowerCase().replace(/^[^a-zA-Z0-9]+/g, '-'),
  snake_case: (s) => s.trim().toLowerCase().replace(/^[^a-zA-Z0-9]+/g, '_'), 'Trim
Whitespace': (s) => s.trim().replace(/\s+/g, ' ')
};

document.querySelectorAll('.transform-btn').forEach(btn => {
  btn.addEventListener('click', () => {
    const operation = btn.dataset.operation; outputBox.textContent =
    transforms[operation](inputBox.value);
  });
});

outputBtn.addEventListener('click', () => {
  navigator.clipboard.writeText(outputBox.textContent);
});

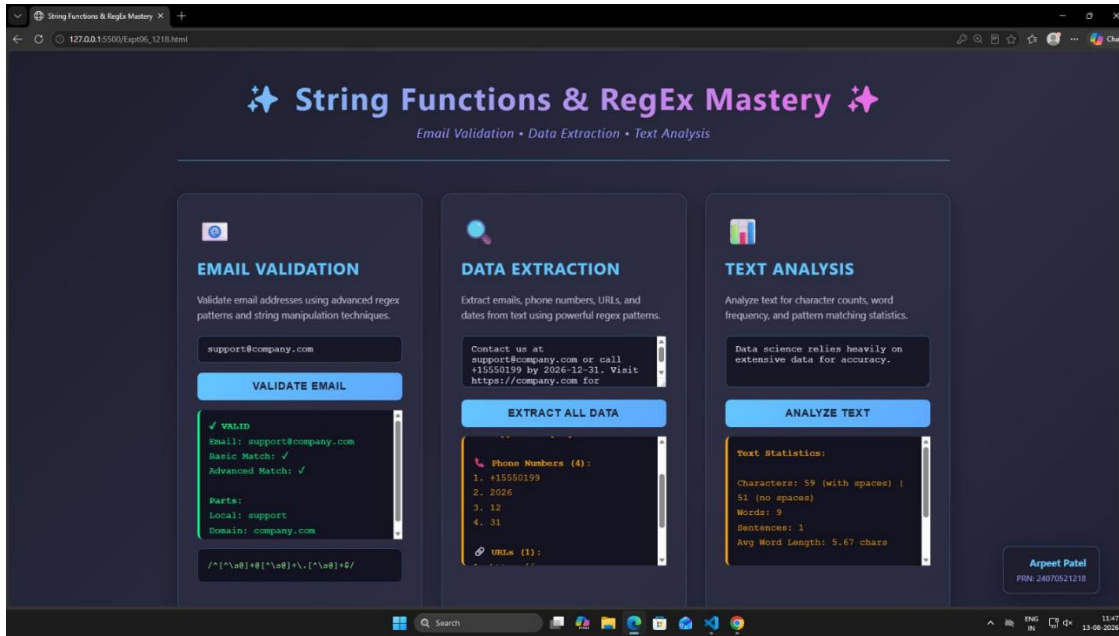
// Regex Studio tab: pattern tester, email validator, extractor
regexTestBtn.addEventListener('click', () => {
  const pattern = new RegExp(regexInput.value, regexFlags.value);
  const matches = Array.from(sampleText.value.matchAll(pattern), m => m[0]);
  regexResult.textContent = matches.length ? matches.join(', ') : 'No matches found';
});
```

Output

Typing or pasting text into the Input String panel live-updates six stat tiles (Characters, Words, Sentences, Lines, Spaces, Read Time). Clicking a Transform Operation button (UPPERCASE, lowercase, Title Case, Reverse, camelCase, kebab-case, snake_case, Trim Whitespace) applies that transformation and shows the result in the Result Output panel, e.g. snake_case on the sample sentence produced

"java_script_is_a_versatile_programming_language_designed_for_interactive_web_applications."

Attach the program output/screenshot.



Student Name: Arpeet Patel

PRN: 24070521218

File Path: Exp6/index.html

Result/Conclusion

This experiment successfully demonstrates live text analytics (character/word/sentence/line counts, reading-time estimation) driven by input events, alongside a suite of string transformation utilities and a regex pattern tester — confirming JavaScript's string and RegExp APIs as effective tools for building interactive text-processing utilities.

Case Study 6

Case Study Title

String Operations & Event-Driven Form Validation

Software/Tools Required

- Web Browser (Chrome, Firefox, Edge)
- HTML5 with CSS3 support
- JavaScript ES6+ (String methods, DOM Events)

Project Description

This case study is split into two parts: (1) String Operations — reversing a paragraph of text, counting vowels, and replacing all occurrences of a word using regex; and (2) Event Handling — a registration form (name, email, phone) validated live on focus, blur, and change events, plus a full validation pass on submit.

Case Study Program Code

```
// 1. String Operations
document.getElementById('reverseBtn').addEventListener('click', () => {
  const text = textInput.value.trim();
  const reversed = text.split('').reverse().join(''); stringOutput.innerHTML =
  '<strong>Reversed String:</strong> ' + reversed;
});

document.getElementById('vowelBtn').addEventListener('click', () => {
```

```

const text = textInput.value.toLowerCase(); const
vowels = text.match(/[aeiou]/g) || [];
stringOutput.innerHTML = '<strong>Vowels Found:</strong> ' + vowels.length;
});

document.getElementById('replaceBtn').addEventListener('click', () => { const
updatedText = textInput.value.replace(/JavaScript/gi, 'JS');
stringOutput.innerHTML = '<strong>Updated Text:</strong><br>' + updatedText;
});

// 2. Event Handling & Validation
const validateEmail = () => {
const value = fieldMap.email.value.trim(); const
pattern = /^[^\s@]+@[^\s@]+\.[^\s@]+$/; return
pattern.test(value);
};

const validatePhone = () => { const
pattern = /^\\d{10}$/;
return pattern.test(fieldMap.phone.value.trim());
};

Object.entries(fieldMap).forEach(([key, field]) => {
field.addEventListener('focus', () => field.closest('.field').classList.add('active'));
field.addEventListener('blur', () => { /* run matching validator */ }); field.addEventListener('change', ()
=> { /* run matching validator */ });
});

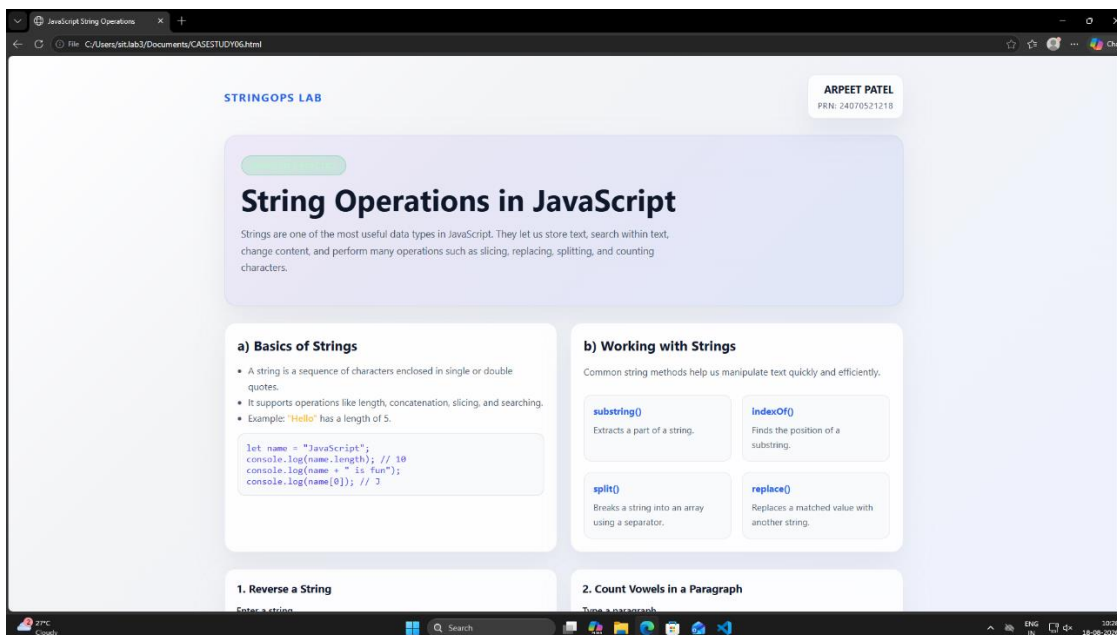
form.addEventListener('submit', (event) => {
event.preventDefault();
const isValid = validateName() && validateEmail() && validatePhone();
formStatus.textContent = isValid ? 'Form submitted successfully!' : 'Please correct the highlighted
fields.';
});
// Full source: CaseStudy/Case Study 6 (24070521218 Arpeet Patel)/index.html

```

Output

The String Operations panel displays the reversed text, vowel count and list, or the text with "JavaScript" replaced by "JS" depending on the button clicked. The registration form highlights each field green (valid) or red (invalid) as the user interacts with it, and shows a success or error banner on submit.

Attach the program output/screenshot.



Student Name: Arpeet Patel

PRN: 24070521218

File Path: Casestudy6/index.html

Result/Conclusion

This case study successfully demonstrates core string manipulation methods (split, reverse, join, match, replace) alongside a full event-driven validation lifecycle (focus, blur, change, submit), confirming JavaScript's ability to deliver responsive, field-level form feedback.

Experiment 7

Experiment Title

Task List Manager using DOM Traversal and Dynamic Element Creation

Software/Tools Required

- Web Browser (Chrome, Firefox, Edge)
- HTML5 with CSS3 support
- JavaScript ES6+ (DOM APIs)

Experiment Program Code

```
const form = document.querySelector("#task-form"); const
input = document.querySelector("#task-input");
const taskSections = document.querySelectorAll(".task-section"); const
taskCount = document.querySelector("#task-count");

function escapeHtml(unsafeText) { return
  unsafeText
    .replace(/&/g, "&amp;")
    .replace(/</g, "&lt;")
    .replace(/>/g, "&gt;")
    .replace(/"/g, "&quot;")
    .replace(/'/g, "&#039;");
}

function updateSummary() {
  const tasks = document.querySelectorAll(".task");
  taskCount.textContent = `${tasks.length} ${tasks.length === 1 ? "task" : "tasks"}`;
  taskSections.forEach((section) => {
    section.querySelector(".section-empty").hidden = section.querySelectorAll(".task").length > 0;
  });
}

function createTask(text) {
  const task = document.createElement("li");
  task.className = "task";
  const safeText = escapeHtml(text);
  task.innerHTML = `
    <span class="task-check" aria-hidden="true"></span>
    <span class="task-text">${safeText}</span>
    <span class="task-actions">
      <select class="status-select"><option value="pending">Pending</option>
        <option value="in-progress">In progress</option>
        <option value="completed">Completed</option></select>
      <button class="edit" type="button">Edit</button>
      <button class="delete" type="button">Delete</button>
    </span>`;
  document.querySelector("#pending-list").append(task);
  updateSummary();
}

form.addEventListener("submit", (event) => {
  event.preventDefault();
  const text = input.value.trim(); if
  (!text) return; createTask(text);
  input.value = "";
});

document.querySelector("section[aria-labelledby='list-title']").addEventListener("click", (event) => { const
  action = event.target.closest("button");
  const task = event.target.closest(".task"); if
  (!action || !task) return;
```

```

if (action.classList.contains("delete")) task.remove(); else if
(action.classList.contains("edit")) {
    const taskText = task.querySelector(".task-text");
    const revisedText = window.prompt("Edit task", taskText.textContent); if
    (revisedText !== null && revisedText.trim()) taskText.textContent =
    escapeHtml(revisedText.trim());
}
updateSummary();
});

document.querySelector("section[aria-labelledby='list-title']").addEventListener("change", (event) => { if
(!event.target.classList.contains("status-select")) return;
const task = event.target.closest(".task");
const destination = document.querySelector(`#${event.target.value}-list`);
task.classList.toggle("completed", event.target.value === "completed");
destination.appendChild(task);
updateSummary();
});

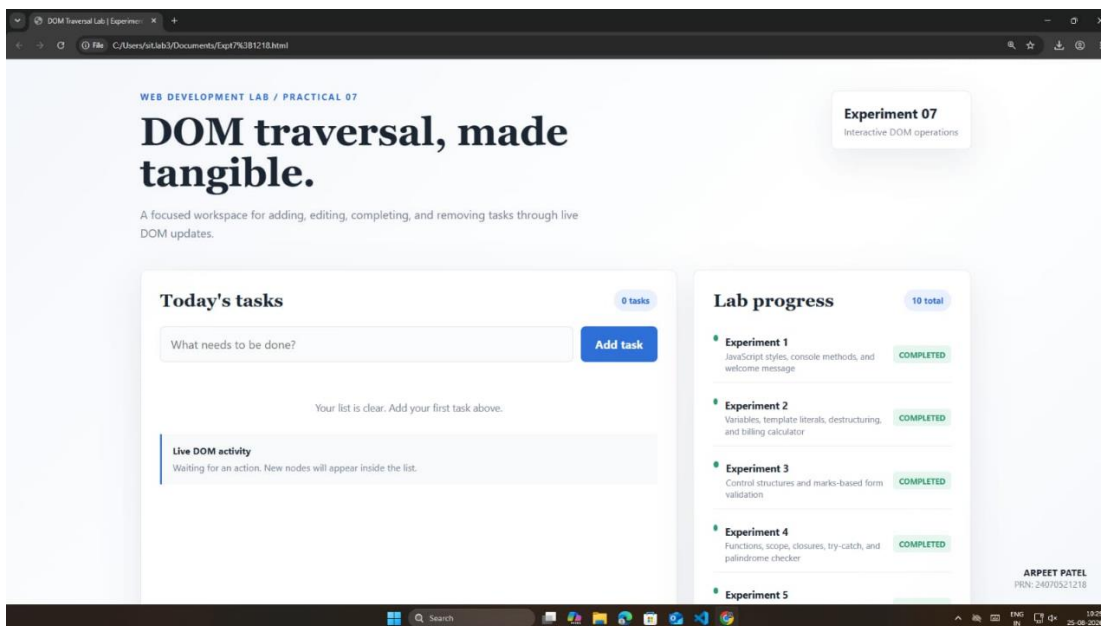
updateSummary();

```

Output

New tasks entered in the input field are appended to a Pending column. Each task row lets the user change its status (Pending/In Progress/Completed) via a dropdown — moving it between the corresponding list sections — edit its text via a prompt dialog, or delete it. A running task count is kept in sync with every change.

Attach the program output/screenshot.



Student Name: Arpeet Patel

PRN: 24070521218

File Path: Exp7/todolist.html

Result/Conclusion

The DOM traversal exercise was successfully implemented, demonstrating createElement()-based node injection, closest()-based event delegation for dynamically added elements, and querySelector-driven traversal between sibling containers — confirming a solid grasp of DOM manipulation beyond static markup.

Case Study 7

Case Study Title

Form Handling with JavaScript — Multi-Field Signup with Password Strength & Theme Toggle

Software/Tools Required

- Web Browser (Chrome, Firefox, Edge)
- HTML5 with CSS3 support
- JavaScript ES6+ (Form Events, Regex)

Project Description

This case study implements a full signup form with dynamically populated Day/Month/Year dropdowns, live password-strength scoring, show/hide password toggles, a light/dark theme switch, and toast-notification feedback, all validated on submit.

Case Study Program Code

```
// Populate Day (1-31) and Year (2026-1950) dropdowns dynamically const
daySelect = document.getElementById('day');
for (let i = 1; i <= 31; i++) {
  const opt = document.createElement('option'); opt.value =
  i; opt.textContent = i; daySelect.appendChild(opt);
}

// Live Password Strength Meter passwordInput.addEventListener('input',
function() {
  const val = this.value; let
  score = 0;
  if (val.length >= 6) score++; if
  (val.length >= 10) score++; if (/[A-
  Z]/.test(val)) score++;
  if (/[0-9]/.test(val)) score++;
  if (/^[A-Za-z0-9]/.test(val)) score++;

  if (score <= 2) { strengthBar.style.width = '33%'; strengthBar.style.backgroundColor = '#ef4444';
  }
    else if (score <= 4) { strengthBar.style.width = '66%'; strengthBar.style.backgroundColor = '#f59e0b'; }
  else { strengthBar.style.width = '100%'; strengthBar.style.backgroundColor = '#10b981'; }
});

// Theme Toggle themeToggleBtn.addEventListener('click',
() => {
  const currentTheme = document.documentElement.getAttribute('data-theme'); const
  newTheme = currentTheme === 'light' ? 'dark' : 'light';
  document.documentElement.setAttribute('data-theme', newTheme);
});

// Submit validation document.getElementById('signupForm').addEventListener('submit',
function (e) {
  e.preventDefault();
  resetErrors();
  const firstname = document.getElementById('firstname').value.trim();
  // ... validates firstname, DOB, username, email, password, etc.
  // shows toast on success, inline errors on failure
});
// Full source: CaseStudy/Case Study 7 (24070521218 Arpeet Patel)/LoginForm.html
```

Output

As the user types a password, a colored strength bar updates live (red/orange/green). The eye icon toggles password visibility. Submitting with missing/invalid fields highlights each one with an inline error; a successful submit shows a toast notification. The theme button switches the whole form between light and dark palettes.

Attach the program output/screenshot.

The screenshot displays a web browser window with the address bar showing '127.0.0.1:5500/casestudy7/LoginForm.html'. The main content is a 'Create Account' form with a title bar 'JavaScript Form Handling'. The form includes a note: '* Mandatory fields must be completed'. The fields are: Firstname (with a person icon and the value 'John'), Lastname (with a person icon and the value 'Doe'), Birthday (with dropdowns for Day, Month, and Year), Username (with an @ icon and the value 'johndoe'), E-mail (with an envelope icon and the value 'john@example.com'), Website (with a globe icon and the value 'https://example.com'), Password (with a shield icon and a strength indicator), and Re-password (with a shield icon and a strength indicator). At the bottom, there is a checkbox labeled 'I agree to the terms & conditions.'.

Student Name: Arpeet Patel

PRN: 24070521218

File Path: casestudy7/LoginForm.html

Result/Conclusion

This case study successfully demonstrates dynamic option population, live input-driven UI feedback (password strength), multi-field submit-time validation, and CSS-variable-based theme switching — combining several JavaScript event-handling patterns in a single realistic signup form.

Experiment 8

Experiment Title

VALENCE Athletic Club — Multi-Field Admission Form with Real-Time Validation

Software/Tools Required

- Web Browser (Chrome, Firefox, Edge)
- HTML5 with CSS3 support
- JavaScript ES6+ (Form validation, Regex)

Experiment Program Code

```
const PLAN_DETAILS = {
  basic: { name: "Basic", duration: "1 Month", price: "₹1,499" }, pro: {
    name: "Pro Athlete", duration: "6 Months", price: "₹6,999" }, elite: { name:
    "Elite VIP", duration: "1 Year", price: "₹11,999" }
};

function validateName() {
  const val = nameInput.value.trim();
  if (val.length === 0) { clearState(nameInput, nameError, nameIndicator); return false; } if
  (val.length >= 3) { setValid(nameInput, nameError, nameIndicator); return true; }
  setInvalid(nameInput, nameError, nameIndicator, "Full name must be at least 3 characters."); return
  false;
}

function validateEmail() {
  const val = emailInput.value.trim();
  const emailPattern = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
  if (val.length === 0) { clearState(emailInput, emailError, emailIndicator); return false; } if
  (emailPattern.test(val)) { setValid(emailInput, emailError, emailIndicator); return true; }
  setInvalid(emailInput, emailError, emailIndicator, "Please enter a valid email address."); return
  false;
}

function validateMobile() {
  mobileInput.value = mobileInput.value.replace(/\D/g, ''); const val =
  mobileInput.value;
  if (val.length === 0) { clearState(mobileInput, mobileError, mobileIndicator); return false; } if
  (val.length === 10) { setValid(mobileInput, mobileError, mobileIndicator); return true; }
  setInvalid(mobileInput, mobileError, mobileIndicator, `Mobile number must be 10 digits
  ${val.length}/10`); return
  false;
}

function validateAge() {
  const val = ageInput.value;
  if (val === "") { clearState(ageInput, ageError, ageIndicator); return false; } const num =
  parseInt(val, 10);
  if (!isNaN(num) && num >= 16 && num <= 100) { setValid(ageInput, ageError, ageIndicator); return true; }
  setInvalid(ageInput, ageError, ageIndicator, "You must be 16 or older to join."); return
  false;
}
// Full source: Experiments/Exp8 (24070521218) Arpeet Patel/exp8html
```

Output

As the user types, each field (name, email, mobile, age, plan) is validated live — showing a green check or a red error message and updating a progress bar. On successful submission, a modal "membership pass" appears summarizing the applicant's name, email, mobile, and selected plan tier with a generated ID.

Attach the program output/screenshot.

Student Name: Arpeet Patel

PRN: 24070521218

File Path: Experiments/Exp8 (24070521218) Arpeet Patel/exp8.html

Result/Conclusion

The admission form successfully demonstrates real-time, field-level validation using regex patterns and numeric range checks, along with dynamic UI feedback (indicators, progress bar, modal). This confirms JavaScript's capability to deliver a responsive, user-friendly validation experience without page reloads.

Case Study 8

Case Study Title

Online Job Application Form — Sectioned Validation with Live Progress Tracking

Software/Tools Required

- Web Browser (Chrome, Firefox, Edge)
- HTML5 with CSS3 support
- JavaScript ES6+ (Form Validation)

Project Description

This case study implements a four-section job application form — Personal Information, Educational Qualification, Professional Information, and Technical Skills — with a live completion progress bar that updates as fields are filled, a theme toggle, and full field-by-field validation on submit.

Case Study Program Code

```

function updateProgress() {
    const requiredInputs = form.querySelectorAll('input[type="text"], input[type="email"], input[type="number"],
input[type="date"], select, textarea');
    let filledCount = 0;
    requiredInputs.forEach(input => { if (input.value.trim() !== '') filledCount++; }); if
(document.querySelector('input[name="gender"]:checked')) filledCount++;
    const percentage = Math.round((filledCount / (requiredInputs.length + 1)) * 100); progressBar.style.width =
percentage + '%';
}
form.addEventListener('input', updateProgress);
form.addEventListener('change', updateProgress);

function validateField(groupId, fieldId, condition) { const
    group = document.getElementById(groupId); const
    errorElem = group.querySelector('.error');
    if (!condition) { group.classList.add('invalid'); if (errorElem) errorElem.style.display = 'flex'; isValid =
false; }
    else { group.classList.remove('invalid'); if (errorElem) errorElem.style.display = 'none'; }
}

form.addEventListener('submit', function (e) {
    e.preventDefault();
    let isValid = true;
    const email = document.getElementById('email').value.trim();
    const emailRegex = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
    validateField('group-email', 'email', emailRegex.test(email));

    const mobile = document.getElementById('mobile').value.trim();
    validateField('group-mobile', 'mobile', /^[0-9]{10}$/.test(mobile));

    const passingYear = parseInt(document.getElementById('passingYear').value, 10);
    validateField('group-passingYear', 'passingYear', passingYear >= 1950 && passingYear <= 2026);
    // ... additional field checks across all four sections

    if (isValid) { successMsg.style.display = 'block'; form.reset(); progressBar.style.width = '0%'; }
});
// Full source: CaseStudy/Case Study 8 (24070521218 Arpeet Patel)/Job_Application_Form.html

```

Output

As fields are filled, the progress bar advances toward 100%. Submitting with any invalid or empty required field highlights that field's group in red with an inline error and blocks submission. When every section passes validation, a success message is shown, the form resets, and the page scrolls to the top.

Attach the program output/screenshot.

File Path: CaseStudy8/Job_Application_Form.html

Result/Conclusion

This case study successfully demonstrates section-by-section validation of a large multi-part form, a reusable validateField() helper pattern, and live progress tracking tied to input/change events — confirming a scalable approach to validating complex forms in vanilla JavaScript.

Experiment 9

Experiment Title

Theme Settings — Persisting User Preference with localStorage and sessionStorage

Software/Tools Required

- Web Browser (Chrome, Firefox, Edge)
- HTML5 with CSS3 support
- JavaScript ES6+ (Web Storage API)

Experiment Program Code

```
() => {
  const sessionTheme = sessionStorage.getItem('preferredTheme'); const
  localTheme = localStorage.getItem('preferredTheme'); const theme =
  sessionTheme || localTheme || 'light';
  document.documentElement.dataset.theme = theme === 'dark' ? 'dark' : 'light';
})();

const themeSelect = document.getElementById('theme-select'); const
statusMessage = document.getElementById('status-message'); const
validThemes = new Set(['light', 'dark']);

function applyTheme(theme) {
  const selectedTheme = validThemes.has(theme) ? theme : 'light';
  document.documentElement.dataset.theme = selectedTheme; themeSelect.value =
  selectedTheme;
}

document.getElementById('save-local').addEventListener('click', () => { const
  theme = themeSelect.value; localStorage.setItem('preferredTheme', theme);
  sessionStorage.removeItem('preferredTheme');
  applyTheme(theme);
  statusMessage.textContent = 'Theme saved using localStorage.';
});

document.getElementById('save-session').addEventListener('click', () => { const theme
  = themeSelect.value; sessionStorage.setItem('preferredTheme', theme);
  localStorage.removeItem('preferredTheme');
  applyTheme(theme);
  statusMessage.textContent = 'Theme saved using sessionStorage.';
});

document.getElementById('clear-preferences').addEventListener('click', () => {
  localStorage.removeItem('preferredTheme'); sessionStorage.removeItem('preferredTheme');
  applyTheme('light');
  statusMessage.textContent = 'Preferences cleared.';
});

const savedTheme = sessionStorage.getItem('preferredTheme') || localStorage.getItem('preferredTheme');
applyTheme(savedTheme || 'light');
if (savedTheme && validThemes.has(savedTheme)) {
  const storageName = sessionStorage.getItem('preferredTheme') ? 'sessionStorage' : 'localStorage';
  statusMessage.textContent = `Theme restored from ${storageName}.`;
}
```

Output

Selecting Light/Dark and clicking "Save using localStorage" persists the choice across browser restarts; "Save using sessionStorage" persists it only for the current tab session. On reload, the page automatically restores the previously saved theme and reports which storage mechanism it came from. "Clear Preferences" resets both stores to Light.

Attach the program output/screenshot.



Student Name: Arpeet Patel

PRN: 24070521218

File Path: Exp9/index.html

Result/Conclusion

The experiment successfully demonstrates the distinction between localStorage (persists indefinitely) and sessionStorage (cleared when the tab closes) for client-side state persistence, along with IIFE-based early theme application to avoid a flash of unstyled content on load.

Case Study 9

Case Study Title

To-Do List with Theme Toggle — Category Tags, Search, Filtering & localStorage Persistence

Software/Tools Required

- Web Browser (Chrome, Firefox, Edge)
- HTML5 with CSS3 support
- JavaScript ES6+ (localStorage, Array methods)

Project Description

This case study builds on the basic to-do list concept by adding category tags (Work/Urgent/Personal/Low), a live search box, All/Active/Completed filtering, a completion progress bar, and persistence of the entire task list to localStorage so it survives page reloads. A light/dark theme toggle is also included.

Case Study Program Code

```
let tasks = JSON.parse(localStorage.getItem('cs9_tasks')) || [
  { id: 1, text: 'Complete JS Lab Experiment 10 submission', category: 'Work', completed: true },
  { id: 2, text: 'Review Case Study 9 Theme Toggle features', category: 'Urgent', completed: false }
];

function saveTasks() { localStorage.setItem('cs9_tasks', JSON.stringify(tasks)); } function

renderTasks() {
  taskList.innerHTML = '';
```

```

const filteredTasks = tasks.filter(task => {
  const matchesFilter = activeFilter === 'all' ? true : activeFilter === 'active' ?
!task.completed : task.completed;
  const matchesSearch = task.text.toLowerCase().includes(searchQuery.toLowerCase()); return
  matchesFilter && matchesSearch;
});
filteredTasks.forEach(task => {
  const li = document.createElement('li');
  li.className = `task-item ${task.completed ? 'completed' : ''}`;
  const badgeClass = task.category === 'Urgent' ? 'badge-urgent' : task.category === 'Work' ?
'badge-work' : 'badge-personal';
  li.innerHTML = `<input type="checkbox" ${task.completed ? 'checked' : ''} data-id="${task.id}">
  <span class="task-text">${escapeHtml(task.text)}</span>
  <span class="task-badge ${badgeClass}">${task.category}</span>`;
  taskList.appendChild(li);
});
updateMetrics();
}

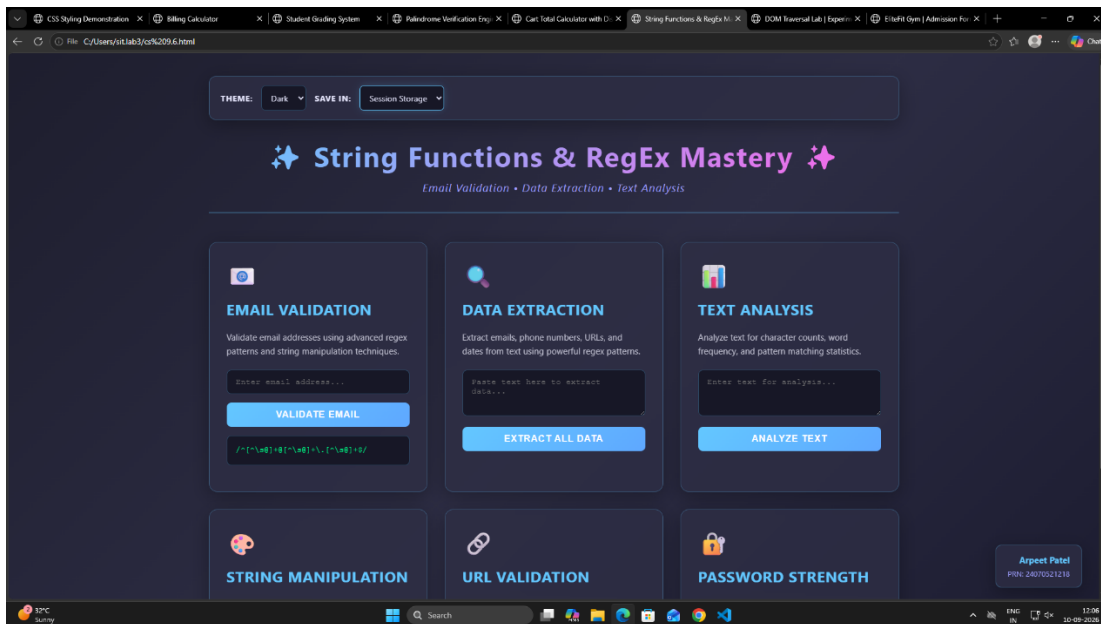
function updateMetrics() { const
total = tasks.length;
const completed = tasks.filter(t => t.completed).length;
const percentage = total === 0 ? 0 : Math.round((completed / total) * 100);
statsText.textContent = `${completed} / ${total} completed (${percentage}%)`;
progressBar.style.width = `${percentage}%`;
}
// Full source: CaseStudy/Case Study 9 (24070521218 Arpeet Patel)/ToDoList_With_Theme_Toggle.html

```

Output

Tasks display with a colored category badge and a checkbox to mark completion (strikethrough style applied). The search box filters the visible list by text match in real time, and the All/Active/Completed buttons narrow it further. A progress bar and "X / Y completed" counter update live. On reload, previously added tasks are restored from localStorage.

Attach the program output/screenshot.



Student Name: Arpeet Patel

PRN: 24070521218

File Path: Casestudy9/ToDoList_With_Theme_Toggle.html

Result/Conclusion

This case study successfully demonstrates persistent client-side state using `localStorage` (surviving page reloads without a backend), combined search-and-filter logic over an array, and category-based UI badging — extending the basic to-do list pattern into a more complete task-management tool.

Experiment 10

Experiment Title

Student Information Portal — Fetching JSON with fetch() and jQuery \$.getJSON()

Software/Tools Required

- Web Browser (Chrome, Firefox, Edge)
- HTML5 with CSS3 support
- JavaScript ES6+ (Fetch API)
- jQuery 3.7.1 (CDN)

Experiment Program Code

```
// Method A: Using Vanilla JavaScript fetch() API function
loadUsingFetch() {
    fetch('students.json')
        .then(response => {
            if (!response.ok) throw new Error(`HTTP error! status: ${response.status}`); return
            response.json();
        })
        .then(data => { renderTable(data); updateMetrics(data); })
        .catch(error => console.error('Fetch error:', error));
}

// Method B: Using jQuery $.getJSON() API function
loadUsingjQuery() {
    $.getJSON('students.json', function(data) {
        renderTable(data); updateMetrics(data);
    }).fail(function(jqxhr, textStatus, error) { console.error('jQuery
    $.getJSON error:', textStatus, error);
    });
}

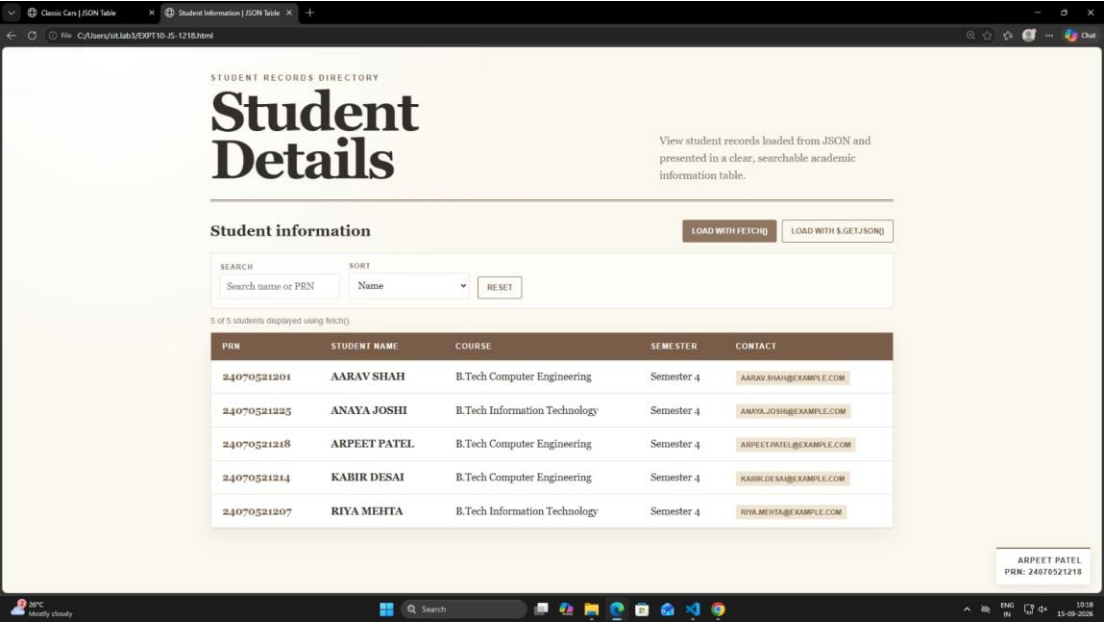
function renderTable(data) { let
    rowsHTML = '';
    data.forEach(student => {
        let gradeClass = 'grade-a';
        if (student.marks < 75 && student.marks >= 60) gradeClass = 'grade-b'; if
        (student.marks < 60) gradeClass = 'grade-c';
        rowsHTML += `<tr>
            <td>${student.id}</td><td>${student.name}</td>
            <td><a href="mailto:${student.email}">${student.email}</a></td>
            <td>${student.course}</td><td>${student.marks}%</td>
            <td><span class="badge-grade ${gradeClass}">${student.grade}</span></td>
            <td>${student.status}</td></tr>`;
    });
    studentTableBody.innerHTML = rowsHTML;
}

// Latency-timed fetch button handler
btnFetchVanilla.addEventListener('click', () => {
    const t0 = performance.now();
    fetch('students.json')
        .then(response => { if (!response.ok) throw new Error('HTTP error ' + response.status); return
        response.json(); })
        .then(data => {
            const latency = (performance.now() - t0).toFixed(2);
            rawStudentsData = data;
            applyFilters();
            statusText.textContent = `Successfully loaded ${data.length} records using fetch() API`;
            latencyBadge.textContent = `Latency: ${latency} ms`;
        });
});
```


Output

Clicking the `fetch()` or jQuery `$.getJSON()` button loads `students.json` and renders a searchable, sortable, course-filterable table of student records (ID, name, email, course, marks, grade badge, status), along with a latency badge showing how long the request took. Both methods produce identical rendered output, illustrating the two approaches to asynchronous JSON retrieval.

Attach the program output/screenshot.



Student Name: Arpeet Patel

PRN: 24070521218

File Path: index.html & students.json

Result/Conclusion

The experiment successfully demonstrates two equivalent approaches to fetching external JSON data — the native Promise-based `fetch()` API and jQuery's `$.getJSON()` — including error handling via `.catch()/.fail()` and dynamic table rendering, confirming both methods integrate cleanly with client-side filtering and sorting.

Case Study 10

Case Study Title

Campus Event Registration System — Dual-Method JSON Fetching with Search, Filter & Diagnostics

Aim

To build a client-side event-registration dashboard that loads student registration records from a JSON source using either the native `fetch()` API or jQuery's `$.getJSON()`, letting the user compare both methods while searching, filtering by status, and viewing live request diagnostics.

Software/Tools Required

- Web Browser (Chrome, Firefox, Edge)
- HTML5 with CSS3 support
- JavaScript ES6+ (Fetch API)
- jQuery 3.7.1 (CDN)

Project Description

The dashboard lets the user pick a load method (fetch or jQuery) via radio buttons, then load real data or deliberately trigger an error (missing_students_data.json) to observe failure handling. A diagnostics panel reports the last request's status, source URL, method used, and latency in milliseconds. Loaded records populate a searchable, status-filterable table with summary stat cards (Total, Registered, Pending, Cancelled), and a persistent light/dark theme toggle is included.

Case Study Program Code

```
// 1. Native JS fetch()
function fetchStudentDataNative(url, startTime) { fetch(url)
    .then(response => {
        if (!response.ok) throw new Error(`HTTP Error ${response.status}: ${response.statusText} || 'File
Not Found'`);
        return response.json();
    })
    .then(data => handleDataLoadSuccess(data, 'JavaScript fetch()', Math.round(performance.now() -
startTime), url))
    .catch(error => handleDataLoadError(error.message, 'JavaScript fetch()', Math.round(performance.now()
- startTime), url));
}

// 2. jQuery $.getJSON()
function fetchStudentDatajQuery(url, startTime) {
$.getJSON(url)
    .done(data => handleDataLoadSuccess(data, 'jQuery $.getJSON()', Math.round(performance.now() -
startTime), url))
    .fail((jqxhr, textStatus, errorThrown) => {
        const status = jqxhr.status ? `HTTP ${jqxhr.status}` : 'Network / Parse Error';
        handleDataLoadError(`Failed to load "${url}": ${status}`, 'jQuery $.getJSON()',
Math.round(performance.now() - startTime), url);
    });
}

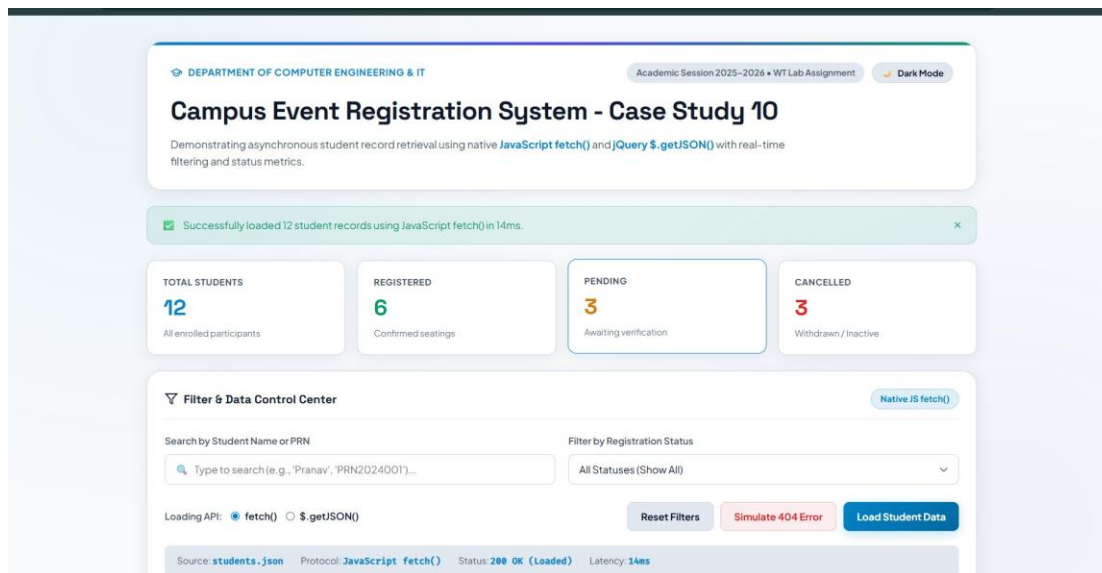
function applyFilters() {
const searchTerm = document.getElementById('searchInput')?.value.trim().toLowerCase() || ''; const
selectedStatus = document.getElementById('statusFilter')?.value || 'All';
const filtered = allStudents.filter(student => {
    const matchesSearch = searchTerm === '' || (student.studentName ||
''.toLowerCase().includes(searchTerm) || (student.prn || '').toLowerCase().includes(searchTerm);
    const matchesStatus = selectedStatus === 'All' || student.registrationStatus === selectedStatus;
    return matchesSearch && matchesStatus;
});
renderTable(filtered);
}

function toggleTheme() {
const nextTheme = currentTheme === 'light' ? 'dark' : 'light';
setTheme(nextTheme);
}
// Full source: CaseStudy/Case Study10 (24070521218 Arpeet Patel)/index.html, script.js, students.json
```

Output

Loading valid data populates the table and stat cards, and shows a green success banner with record count, method, and latency. Triggering the error case shows a red error banner and empty-state message. The search box and status dropdown filter the table live, and "Clear Filters" resets both.

Attach the program output/screenshot.



Student Name: Arpeet Patel

PRN: 24070521218

File Path: CaseStudy10/index.html

Result/Conclusion

This case study successfully demonstrates both major approaches to asynchronous JSON retrieval (fetch vs. jQuery), explicit success/error branching with user-facing diagnostics, and combined search-and-filter logic over fetched data — providing a realistic, comparative view of client-side data loading strategies.